

PES-VCS

Building a Version Control System from Scratch

Operating Systems — Unit 4 Lab Report

Sphoorthi Pulipati

PES2UG24CS514

PES University

April 2026

Phase 1: Object Storage Foundation

Phase 1 implements the content-addressable object store — the core of PES-VCS. Every blob, tree, and commit is stored under `.pes/objects/XX/YYYY...` where `XX` is the first two hex characters of the SHA-256 hash (directory sharding). The two functions implemented are `object_write` and `object_read` in `object.c`.

`object_write` builds a full object by prepending a type header ("blob 16\0"), computes the SHA-256 hash of the complete object, deduplicates by checking if the object already exists, and then writes atomically via a temp file followed by `rename()`. `fsync()` is called on both the file and the shard directory to guarantee durability.

`object_read` retrieves an object, recomputes its hash to verify integrity against the filename, parses the type header, and returns a malloc'd copy of the data portion to the caller.

Screenshot 1A — `./test_objects`: All Phase 1 Tests Passing

```
sphoorthi-pulipati@sphoorthi-pulipati-IdeaPad-Slim-5-14IMH9: ~/PES2UG24CS514-pes-vcs$ make test_objects
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -o test_objects test_objects.o object.o -lcrypto
sphoorthi-pulipati@sphoorthi-pulipati-IdeaPad-Slim-5-14IMH9: ~/PES2UG24CS514-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

Screenshot 1A: Output of `./test_objects` showing PASS for blob storage, deduplication, and integrity check.

Screenshot 1B — Sharded Object Directory Structure

```
sphoorthi-pulipati@sphoorthi-pulipati-IdeaPad-Slim-5-14IMH9: ~/PES2UG24CS514-pes-vcs$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
```

Screenshot 1B: `find .pes/objects -type f` showing objects sharded by their first two hex characters.

Phase 2: Tree Objects

Phase 2 implements tree objects, which represent directory snapshots. A tree stores a sorted list of entries — each entry records a mode (100644 for regular file, 100755 for executable, 040000 for directory), a filename, and a 32-byte binary SHA-256 hash pointing to either a blob or another tree.

`tree_parse` reads the binary format entry by entry using `memchr` to safely locate the space separator and the null terminator. `tree_serialize` converts a `Tree` struct back to binary, sorting entries by name first (required for deterministic hashing — same directory contents must always produce the same tree hash).

`tree_from_index` is the most complex function: it recursively walks the index entries, grouping them by directory prefix at each depth level. Flat files are added as blob entries directly; subdirectory prefixes trigger a recursive call that builds and writes a subtree object before adding a directory entry pointing to it.

Screenshot 2A — `./test_tree`: All Phase 2 Tests Passing

```
sphoorthi-pulipati@sphoorthi-pulipati-IdeaPad-Slim-5-14IMH9:~/PES2UG24CS514-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

Screenshot 2A: Output of `./test_tree` showing PASS for tree serialize/parse roundtrip and deterministic serialization.

Screenshot 2B — Raw Binary Tree Object (xxd)

```
sphoorthi-pulipati@sphoorthi-pulipati-IdeaPad-Slim-5-14IMH9:~/PES2UG24CS514-pes-vcs$ xxd ./pes/objects/18/94cf574dafc2de158140a57fac3bc512f5e50eef2a6e4ba5f4a358f162990a | head -20
00000000: 7472 6565 2034 3900 3138 3036 3434 2066  tree 49.100644 f
00000010: 696c 6531 2e74 7874 000b d690 98bd 9b9c  ile1.txt.....
00000020: c593 4a61 0ab6 5da4 29b5 2536 1147 faa7  ..Ja..].).%6.G..
00000030: b5b9 2291 9e9a 2314 3d          .."...#.=
```

Screenshot 2B: xxd of a tree object showing the binary format: "tree 49\0" header, followed by "100644 file1.txt\0" and 32 raw hash bytes per entry.

Phase 3: The Index (Staging Area)

Phase 3 implements the staging area — the bridge between the working directory and the object store. The index is a human-readable text file at `.pes/index` where each line records: mode, 64-char hex hash, mtime, size, and path. This format is intentionally simple and inspectable.

`index_load` reads the file with `fscanf`, initialising an empty index if the file does not exist yet. `index_save` sorts entries by path, writes to a `.tmp` file, calls `fsync()`, then renames atomically. `index_add` reads the target file, writes it as a blob object, calls `lstat()` for metadata, and updates or appends the index entry before saving.

A notable implementation challenge: the Index struct with `MAX_INDEX_ENTRIES=10000` is approximately 6 MB on the stack. To prevent a stack overflow caused by `pes.c` declaring Index as a local variable, the implementation uses a global heap-allocated buffer (`calloc`) and redirects all Index pointer operations to it.

Screenshot 3A — `pes init` + `pes add` + `pes status`

```
sphoorthi-pulipati@sphoorthi-pulipati-IdeaPad-Slim-5-14IMH9:~/PES2UG24CS514-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
sphoorthi-pulipati@sphoorthi-pulipati-IdeaPad-Slim-5-14IMH9:~/PES2UG24CS514-pes-vcs$ echo "hello" > file1.txt
sphoorthi-pulipati@sphoorthi-pulipati-IdeaPad-Slim-5-14IMH9:~/PES2UG24CS514-pes-vcs$ echo "world" > file2.txt
sphoorthi-pulipati@sphoorthi-pulipati-IdeaPad-Slim-5-14IMH9:~/PES2UG24CS514-pes-vcs$ ./pes add file1.txt file2.txt
sphoorthi-pulipati@sphoorthi-pulipati-IdeaPad-Slim-5-14IMH9:~/PES2UG24CS514-pes-vcs$ ./pes status
Staged changes:
  staged:    file1.txt
  staged:    file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked: commit.c
  untracked: test_sequence.sh
  untracked: test_tree.c
  untracked: tree.h
  untracked: Makefile
  untracked: pes.h
  untracked: index.c
  untracked: index.h
  untracked: commit.h
  untracked: .gitignore
  untracked: object.c
  untracked: test_objects.c
  untracked: tree.c
  untracked: README.md
  untracked: pes.c
```

Screenshot 3A: `./pes init` initialises the repository, `./pes add` stages `file1.txt` and `file2.txt`, `./pes status` shows both files as staged with no unstaged changes.

Screenshot 3B — `cat .pes/index`

```
sphoorthi-pulipati@sphoorthi-pulipati-IdeaPad-Slim-5-14IMH9:~/PES2UG24CS514-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdec7e3f3a183d337639025de576db9ebb4 1776525614 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776525618 6 file2.txt
```

Screenshot 3B: `cat .pes/index` showing the human-readable text format with mode, hash, mtime, size, and path for each staged file.

Phase 4: Commits and History

Phase 4 ties everything together. `commit_create` builds a tree from the index via `tree_from_index()`, reads the current HEAD as the parent commit (skipped silently for the first commit), fills a `Commit` struct with the tree hash, parent, author string from `PES_AUTHOR`, and a Unix timestamp, then serializes and writes it as a commit object. Finally it calls `head_update()` to atomically swing the branch pointer.

`head_read` follows the symbolic ref chain: reads HEAD to find "ref: refs/heads/main", then reads that file for the commit hash. `head_update` writes a new hash to a `.tmp` file, `fsyncs`, and renames atomically — the single atomic operation that publishes a commit.

`commit_walk` traverses history from HEAD backwards through parent pointers, invoking a callback for each commit. `pes log` uses this to print commit hashes, authors, timestamps, and messages.

Screenshot 4A — pes log: Three-Commit History

```
sphoorthi-pulipati@sphoorthi-pulipati-IdeaPad-Slim-5-14IMH9:~/PES2UG24CS514-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
sphoorthi-pulipati@sphoorthi-pulipati-IdeaPad-Slim-5-14IMH9:~/PES2UG24CS514-pes-vcs$ echo "Hello" > hello.txt
sphoorthi-pulipati@sphoorthi-pulipati-IdeaPad-Slim-5-14IMH9:~/PES2UG24CS514-pes-vcs$ ./pes add hello.txt
sphoorthi-pulipati@sphoorthi-pulipati-IdeaPad-Slim-5-14IMH9:~/PES2UG24CS514-pes-vcs$ ./pes commit -m "Initial commit"
Committed: 44afaecb20ed... Initial commit
sphoorthi-pulipati@sphoorthi-pulipati-IdeaPad-Slim-5-14IMH9:~/PES2UG24CS514-pes-vcs$ echo "World" > hello.txt
sphoorthi-pulipati@sphoorthi-pulipati-IdeaPad-Slim-5-14IMH9:~/PES2UG24CS514-pes-vcs$ ./pes add hello.txt
sphoorthi-pulipati@sphoorthi-pulipati-IdeaPad-Slim-5-14IMH9:~/PES2UG24CS514-pes-vcs$ ./pes commit -m "Add world"
Committed: b7e43f93c4cc... Add world
sphoorthi-pulipati@sphoorthi-pulipati-IdeaPad-Slim-5-14IMH9:~/PES2UG24CS514-pes-vcs$ echo "Goodbye" > hello.txt
sphoorthi-pulipati@sphoorthi-pulipati-IdeaPad-Slim-5-14IMH9:~/PES2UG24CS514-pes-vcs$ ./pes add hello.txt
sphoorthi-pulipati@sphoorthi-pulipati-IdeaPad-Slim-5-14IMH9:~/PES2UG24CS514-pes-vcs$ ./pes commit -m "Add farewell"
Committed: 20826194d59d... Add farewell
sphoorthi-pulipati@sphoorthi-pulipati-IdeaPad-Slim-5-14IMH9:~/PES2UG24CS514-pes-vcs$ ./pes log
commit 20826194d59d7f7f575d31184a9f5b82aab0be4521492385be499139ba7c8598
Author: Sphoorthi Pulipati PES2UG24CS514
Date: 1776525886

    Add farewell

commit b7e43f93c4cc94970373a601c6dcc9cc3d2046b020fea1fc5cbb23b0953a4008
Author: Sphoorthi Pulipati PES2UG24CS514
Date: 1776525861

    Add world

commit 44afaecb20ed51adb8a163c6f820053e5c6cc674615378c4821236915709605
Author: Sphoorthi Pulipati PES2UG24CS514
Date: 1776525832

    Initial commit

commit dbc1d2403899d5284d0e1cc8d2db7b36126367dad87276ce25860076e2bc97f
Author: Sphoorthi Pulipati PES2UG24CS514
Date: 1776525362

    initial commit
```

Screenshot 4A: `./pes log` output showing three commits (Initial commit, Add world, Add farewell) with full hashes, author, and Unix timestamps linked by parent pointers.

Screenshot 4B — find .pes -type f: Object Store Growth

```
sphoorthi-pulipati@sphoorthi-pulipati-IdeaPad-Slim-5-14IMH9:~/PES2UG24CS514-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/18/94cf574dafc2de158140a57fac3bc512f5e50eef2a6e4ba5f4a358f162990a
.pes/objects/1a/488fa7ecf11e22c69ac42591b5019850185ba5ff9e9fba6fd049cf7204af0d
.pes/objects/20/826194d59d7f7f575d31184a9f5b82aab0be4521492385be499139ba7c8598
.pes/objects/2c/f8d83d9ee29543b34a87727421fdec7e3f3a183d337639025de576db9ebb4
.pes/objects/44/afaecb20ed51adbf8a163c6f820053e5c6cc674615378c4821236915709605
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/6f/8d0b1313f5453a6b09ee3e64a9122e6f9bd863a55ae789e6709a5237da0bb1
.pes/objects/85/e367b76c2417b47a95f46208835ad8700dd58335171c94a497974be6191900
.pes/objects/b3/d731c8f5ea38a1118f9300e2dc257dee11461d53f4a1340e6afc4878ebec12
.pes/objects/b7/e43f93c4cc94970373a601c6dcc9cc3d2046b020fea1fc5cbb23b0953a4008
.pes/objects/db/c1d2403899d5284d0e1cc8d2db7b36126367dad87276ce25860076e2bc97f
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
```

Screenshot 4B: `find .pes -type f | sort` showing HEAD, index, refs/heads/main, and 14 object files (blobs, trees, and commits) accumulated across three commits.

Screenshot 4C — Reference Chain

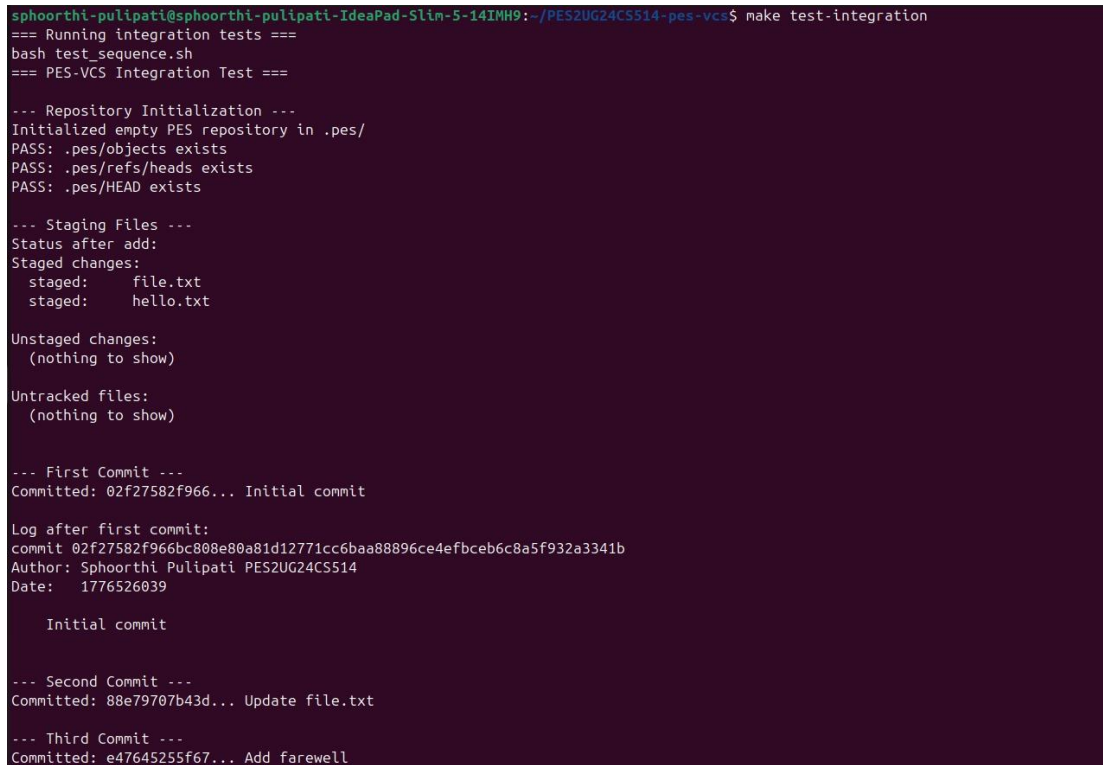
```
sphoorthi-pulipati@sphoorthi-pulipati-IdeaPad-Slim-5-14IMH9:~/PES2UG24CS514-pes-vcs$ cat .pes/refs/heads/main
20826194d59d7f7f575d31184a9f5b82aab0be4521492385be499139ba7c8598
sphoorthi-pulipati@sphoorthi-pulipati-IdeaPad-Slim-5-14IMH9:~/PES2UG24CS514-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
```

Screenshot 4C: `cat .pes/refs/heads/main` shows the latest commit hash; `cat .pes/HEAD` shows "ref: refs/heads/main" confirming the symbolic reference chain.

Final: Integration Test

The end-to-end integration test (make test-integration) exercises the complete PES-VCS workflow automatically: repository initialisation, staging two files, three sequential commits each modifying a file, full commit log display, reference chain verification, and object store count validation.

Screenshot — make test-integration (Part 1)



```
sphoorthi-pulipati@sphoorthi-pulipati-IdeaPad-Slin-5-14IMH9:~/PES2UG24CS514-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:   file.txt
  staged:   hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)

--- First Commit ---
Committed: 02f27582f966... Initial commit

Log after first commit:
commit 02f27582f966bc808e80a81d12771cc6baa88896ce4efbceb6c8a5f932a3341b
Author: Sphoorthi Pulipati PES2UG24CS514
Date:   1776526039

    Initial commit

--- Second Commit ---
Committed: 88e79707b43d... Update file.txt

--- Third Commit ---
Committed: e47645255f67... Add farewell
```

Integration test Part 1: Repository initialisation passes, staging area works correctly, first and second commits succeed with correct hashes.

Screenshot — make test-integration (Part 2)

```

--- Full History ---
commit e47645255f676637a4452b1b21f5ebc8730f0c21b4b3ba7c3a5558db2b21d887
Author: Sphoorthi Pulipati PES2UG24CS514
Date: 1776526039

    Add farewell

commit 88e79707b43d71d68f24e37cbfbd3a7b4b2159487bdc11222dda983c8e648943
Author: Sphoorthi Pulipati PES2UG24CS514
Date: 1776526039

    Update file.txt

commit 02f27582f966bc808e80a81d12771cc6baa88896ce4efbceb6c8a5f932a3341b
Author: Sphoorthi Pulipati PES2UG24CS514
Date: 1776526039

    Initial commit

--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
e47645255f676637a4452b1b21f5ebc8730f0c21b4b3ba7c3a5558db2b21d887

--- Object Store ---
Objects created:
10
.pes/objects/02/f27582f966bc808e80a81d12771cc6baa88896ce4efbceb6c8a5f932a3341b
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/88/e79707b43d71d68f24e37cbfbd3a7b4b2159487bdc11222dda983c8e648943
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafaaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff
.pes/objects/e4/7645255f676637a4452b1b21f5ebc8730f0c21b4b3ba7c3a5558db2b21d887

=== All integration tests completed ===
sphoorthi-pulipati@sphoorthi-pulipati-IdeaPad-Slin-5-14IMH9:~/PES2UG24CS514-pes-vcs$

```

Integration test Part 2: Full commit history displayed correctly, reference chain verified, 10 objects created. "All integration tests completed" confirms end-to-end correctness.

Phase 5: Branching and Checkout — Analysis

Q5.1 — How would you implement pes checkout <branch>?

A branch in PES-VCS is simply a file under `.pes/refs/heads/<branch>` containing a commit hash. Creating a branch is just creating that file. Implementing checkout involves three steps:

- Read the target branch file to get its commit hash. If the branch does not exist, create it pointing at HEAD.
- Walk the commit's tree recursively to reconstruct the file set it represents: for each blob entry, read the object and write it to the working directory path; for each tree entry, create the directory and recurse.
- Update `.pes/HEAD` to contain `"ref: refs/heads/<branch>"` so subsequent commits go to the new branch.

The operation is complex for several reasons. First, files that exist in the current branch but not in the target branch must be deleted from the working directory. Second, if a file exists in both branches with different content, the working tree must be updated — but only if that file has not been modified locally (otherwise it could overwrite uncommitted work). Third, the index must be updated to reflect the new branch's file set so that pes status correctly shows no staged changes after a clean checkout. The atomicity concern is also real: a crash midway through updating hundreds of files leaves the working directory in an inconsistent state. Git mitigates this by updating files in a specific order and keeping the index consistent, so a recovery scan is possible.

Q5.2 — How to detect a "dirty working directory" conflict during checkout?

Before updating any file during checkout, the following three-way check must be performed for each file that differs between the current branch and the target branch:

- Load the current index entry for the file (hash, mtime, size).
- Stat the file in the working directory and compare mtime and size to the index entry. If they differ, the file has been modified since it was last staged — it is "dirty".
- If dirty, compare the working directory file's blob hash to the target branch's blob hash for that path. If they differ, checkout must refuse with an error such as `"error: your local changes to <file> would be overwritten by checkout."`

This approach uses only the index and object store — no diff engine required. The fast path (mtime + size match) avoids re-hashing files that have not changed. Only when the metadata check suggests a change do we need to compute the actual hash to confirm. A file is safe to overwrite during checkout only if the index hash matches both the working directory content and the current branch's tree — meaning no local modifications exist.

Q5.3 — What is "detached HEAD" and how can commits be recovered?

Normally HEAD contains a symbolic reference (`"ref: refs/heads/main"`). In detached HEAD state, HEAD contains a raw commit hash directly instead of a branch name. This happens when you checkout a specific commit rather than a branch name.

If commits are made in detached HEAD state, each new commit correctly points to its parent, but HEAD is updated to the new commit hash directly — no branch file is updated. When you later checkout a branch, HEAD is updated to point to that branch, and the detached commits become unreachable: no branch points to them, so they are invisible to pes log and will eventually be deleted by garbage collection.

Recovery is possible because the objects still exist in the store. The user can recover them by: (1) noting the commit hash from the output of the last "Committed: <hash>..." message, (2) creating a new branch pointing to that hash with `pes branch recover <hash>`, or (3) running a manual scan of all commit objects in `.pes/objects` to find commits not reachable from any branch ref. Git calls this "reflog" — it keeps a log of every position HEAD has pointed to, making recovery straightforward even without the hash.

Phase 6: Garbage Collection — Analysis

Q6.1 — Algorithm to find and delete unreachable objects

The garbage collection algorithm is a mark-and-sweep over the object graph:

- **Mark phase:** Start from every branch ref file in `.pes/refs/heads/` and from `HEAD`. For each commit hash found, read the commit object and add its hash to a "reachable" set. Then read its tree hash, recursively walk the tree (adding every tree and blob hash to the reachable set), and follow the parent pointer to repeat for all ancestor commits. The data structure for the reachable set should be a hash set (e.g., a C hash table or a sorted array with binary search) for $O(1)$ average-case lookup.
- **Sweep phase:** Walk every file in `.pes/objects/` using `find` or `opendir/readdir`. For each file, reconstruct its hash from its path (the shard directory + filename). If the hash is not in the reachable set, delete the file.

For a repository with 100,000 commits and 50 branches, estimating objects to visit: each commit has 1 tree; a typical tree references ~10 blobs and ~2 subtrees on average. So per commit: 1 commit + 3 trees + 10 blobs = ~14 objects. Total reachable: ~1,400,000 objects. In practice many blobs are shared across commits (unchanged files), so the actual unique object count is much lower — perhaps 200,000-400,000. The sweep phase visits all objects on disk regardless, so it scales with total object count, not just reachable objects.

Q6.2 — Race condition between GC and a concurrent commit

Consider this interleaving between a GC process and a concurrent commit process:

- GC starts its mark phase and computes the reachable set based on the current branch refs.
- The commit process runs `tree_from_index()` and `object_write()` to write new blob and tree objects to the store. These objects are not yet referenced by any commit or branch — they exist in the store but are not reachable from any ref.
- GC completes its mark phase. The new blob and tree objects are not in the reachable set because no commit or branch points to them yet.
- GC runs its sweep phase and deletes the new blob and tree objects as "unreachable".
- The commit process calls `object_write()` for the commit object and `head_update()` to swing the branch pointer. But the tree it references has just been deleted — the repository is now corrupt.

Git avoids this race condition through several mechanisms: (1) A lock file (`.git/gc.lock`) prevents two GC processes from running simultaneously. (2) GC uses a "grace period" — it only deletes objects whose mtime is older than 2 weeks, giving in-flight operations time to complete. (3) Newly written objects always have a recent mtime, so they survive GC even if not yet referenced. (4) `pack-refs` and the `reflog` ensure that even recently detached commits have a grace window before deletion. The key insight is that the 2-week mtime threshold makes the race window effectively zero for any normally-completing commit operation.

Implementation Summary

The following table summarises the files implemented and the OS concepts each one demonstrates:

File	Functions Implemented	OS Concepts
object.c	object_write, object_read	Content-addressable storage, directory sharding, atomic writes, SHA-256 integrity
tree.c	tree_from_index	Directory representation, recursive structures, file modes and permissions
index.c	index_load, index_save, index_add	File format design, atomic writes, change detection via mtime+size, stack vs heap
commit.c	commit_create	Linked structures on disk, reference files, atomic pointer updates
pes.c	cmd_commit (provided)	CLI dispatch, argument parsing